Sealstone

Solution Proposal and Architecture Report

Responding to problem statement **B-03, MilestoneVault: Adversarial Smart-Contract Escrow**. The Solidity contract keeps the booklet's name, `MilestoneVault`. Sealstone is the name of the product built around it.

Date	11 October 2026
Team	Omkar Rane (builder) and a teammate (research, test scenarios, pitch)
Stage	Working prototype on a local test chain. All data is fictional and no real funds are involved.
Source of figures	Test counts and results are copied from `docs/PROGRESS.md` as recorded on this date. They were not re-run when this report was produced.

Contents

1 Summary

2 The problem we were given

3 Proposed solution

4 System architecture

5 Contract design

6 Security design

7 The AI component

8 Verification evidence

9 Requirements traceability

10 Project status

11 Limitations and known issues

12 Roadmap and open decisions

13 Related work

A Commands and repository layout

1 Summary

Milestone payments are common in freelance work, construction and grant funding, and they go wrong in three predictable ways. The agreement is written in prose that two people read differently. The file that gets approved is not the file that was reviewed. Or the money sits with an intermediary whose rules nobody can inspect. Sealstone is an escrow built to close each of these gaps, and to let anyone test that it has.

We organised the product around three seals.

Seal	What it guarantees	How it is enforced
Terms	The agreement cannot be funded until every unclear term has been settled by a person, and neither side can later claim different words.	The confirmed checklist is reduced to canonical JSON and hashed with keccak256. Both the client and the provider call `sign(hash)` on the contract before any money moves.
Work	The client approves one specific deliverable. A different file cannot be approved in its place.	The provider records the SHA-256 of the deliverable when marking a milestone complete. `approve` takes a hash and reverts unless it matches the stored one.
Money	Funds move only through defined transitions, only for the party allowed to trigger them, and the books always balance.	A role-gated state machine with pull payments, a reentrancy guard and a live reconciliation view, backed by 138 contract tests and an Attack Lab that fires real adversarial transactions.

Artificial intelligence has a narrow, deliberate job. It reads the agreement text and proposes a checklist, flagging terms that look ambiguous. It does not decide anything about money. A deterministic validator re-checks every figure and party, a person must resolve each flagged item, and only the confirmed result can be written to the chain. The booklet asks for exactly this boundary, and we kept it visible in the code as a type distinction between a proposed and a confirmed checklist.

The prototype runs entirely on a local Hardhat chain with a rules-based fallback, so the demonstration works without a network connection. Where this report cites test figures they come from the project's progress log, and where something has not been built yet we say so. Sections 10 and 11 list what remains open.

2 The problem we were given

The VORTEXA 3.0 booklet describes problem B-03 as follows. We checked the text below against the booklet PDF on 11 October 2026.

> **The Challenge.** Build a milestone-based escrow in which simulated funds are released or refunded only when the agreement's conditions are met. The agreement names a client, a provider, and an arbiter who resolves disputes, and defines each milestone's amount and approval conditions.

The booklet then lists five requirements, which we paraphrase here.

1. Implement the logic as a Solidity contract tested on a local Hardhat network. No real cryptocurrency and no public deployment are required.
2. Enforce the lifecycle. The provider marks a milestone complete, the client approves it or disputes it, and the arbiter releases or refunds disputed funds. Each step belongs to one party and undefined transitions are rejected.
3. Prevent duplicate releases, unauthorised refunds and out-of-order transitions, and keep the contract balance reconciled with deposits and payments.
4. Back each guarantee with automated tests, including a second release attempt, an unauthorised refund, an out-of-order transition and a re-entrancy attack.
5. AI may turn agreement text into a proposed checklist with ambiguities. A person must confirm it, ambiguities must be resolved, and AI output must never be written to the contract directly.

The booklet's common rules for every team also apply. The deliverable must be a working MVP that judges can test live. AI must be used meaningfully, which excludes a chatbot or a call that only rewrites text. Where a decision has real consequences, the outcome must follow explicit, checkable rules, and AI "must not silently override them". Data must be fictional, with no real personal records, real financial transactions or live third-party systems. External AI may be used, but "the core demonstration must not depend on them; provide a local fallback."

Where we chose to be stricter than the booklet

Two of our working rules go further than the text. We require the full demo to run with the network switched off, whereas the booklet asks only for a local fallback. We also avoid public networks by default, whereas the booklet says a public deployment is not required. We treat both as our own standards. The booklet publishes no judging rubric, so we have not assumed any scoring weights.

Why this is difficult

Code that holds money has no undo. A payment released twice cannot be recovered, a missing access check lets the wrong person take funds, and a re-entrant call can drain a balance before the contract updates its own records. The text side is just as unforgiving, because a sentence such as "the remaining funds after the first two milestones" leaves the amount undefined at the moment it matters. A good solution has to handle both halves, and the two meet at the point where prose becomes numbers on a ledger.

3 Proposed solution

3.1 Three layers, one flow

Sealstone treats the agreement, the deliverable and the funds as three separate things that can each be tampered with, and protects each one separately.

Agreement integrity. The extractor proposes a checklist and a list of ambiguities. A shared validator then checks parties, arithmetic and wording on its own, and can add or escalate flags but never remove one. A person resolves every flag. The confirmed checklist is serialised as canonical JSON, hashed, and stored in the contract as an immutable value. Neither party can fund until both have signed that same hash.

Deliverable integrity. When the provider marks a milestone complete they submit a SHA-256 hash of the deliverable. To release the money the client must present that exact hash. If a different file has been swapped in, the call reverts with `DeliverableHashMismatch` and shows both hashes.

Fund integrity. Payments are credited to a balance and withdrawn by the recipient (a pull payment). The state machine rejects every undefined transition with a named error. A reconciliation view compares the contract's real balance with what its own records say it should hold, and reports any ether that was force-sent from outside as surplus that can never be spent.

3.2 The demonstration scenario

The built-in scenario is fictional. A campus incubation cell funds a student team, KrishiLens, which is building a crop-disease photo app, with a 5 ETH grant across three milestones. A faculty mentor acts as arbiter and is fixed when the contract is deployed. The figures below are taken from the seeded checklist, `samples/krishilens.confirmed.json`.

No.	Milestone	Amount	Deadline	Deliverable
M1	Problem validation report	1.0 ETH	31 Oct 2026	Document
M2	Working prototype	2.5 ETH	15 Dec 2026	Code
M3	Pilot with farmers	1.5 ETH	None given	Document

The source agreement text was written to contain real ambiguities: a payment tied to "the mentor's satisfaction", an amount described only as the remaining funds, a bonus clause that makes the total unclear, and a final milestone with no date. The rules extractor raises five flags on this text. A live language model run raised more, as section 7 describes.

3.3 User flow

1. Paste or load an agreement. The service proposes a checklist and lists ambiguities, each with the sentence that caused it.
2. Resolve every ambiguity. Blocking items must be edited or the clause removed. A warning can be acknowledged. The Confirm button stays locked until the gate is satisfied.
3. Confirm. The checklist is hashed and the contract is deployed with the parties, the milestone amounts and the agreement hash.

4. The client and then the provider each call `sign(hash)`. These are ordinary on-chain transactions, not off-chain signatures.
5. The client funds the exact total.
6. The provider submits milestone one with the SHA-256 of the file. The client approves with that hash, or disputes with a reason. If the client does not respond within the review window the provider may escalate.
7. For a disputed milestone the arbiter releases or refunds. Released money is credited to the provider and refunded money to the client, and each withdraws their own credit.
8. When the last milestone reaches a final state the agreement closes. At any point anyone can open the vault page, see the ledger reconcile and re-hash the stored terms against the chain.

3.4 Interface

The web application has three working areas. The agreement wizard handles steps one to five. The vault page shows each milestone with the actions the current role may take, an activity timeline built from contract events, a ledger panel, and a Verify panel that re-hashes the terms in the browser and compares the result with the chain. The Attack Lab, described in section 8, lets a visitor run adversarial transactions and see the contract's response. Roles are switched with Hardhat's unlocked test accounts, so there is no wallet extension in this prototype.

4 System architecture

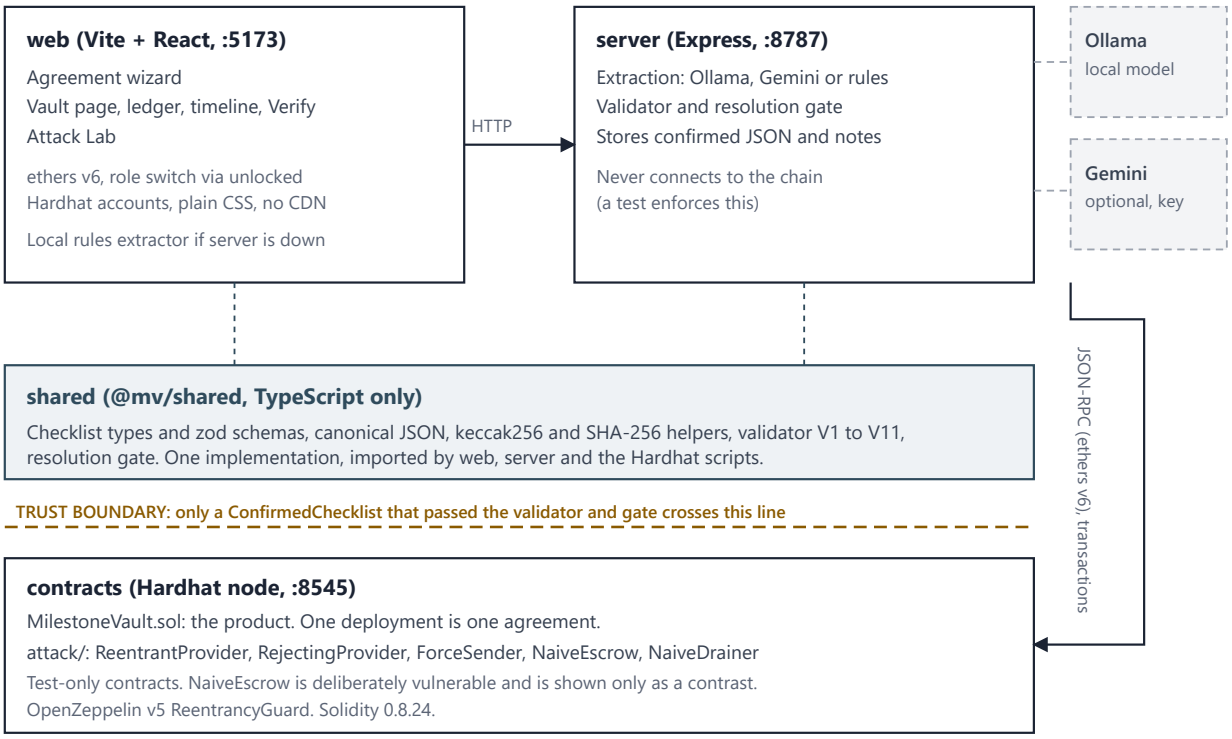


Figure 1. Components and the boundary between proposed and confirmed data. The web client talks to the chain directly; the server handles language work and storage only.

4.1 Components

Package	Technology	Responsibility
`contracts/`	Hardhat 2, Solidity 0.8.24, OpenZeppelin v5, TypeScript tests, solidity-coverage	The escrow contract, the attacker contracts, the test suites, the deploy and seed scripts, and the ABI export used by the web app.
`server/`	Node 20 or later, Express, TypeScript, zod	Extraction with provider selection and fallback, the validate and gate-check routes, confirmation, and storage of confirmed agreements and evidence notes. It never touches the chain.
`web/`	Vite, React, TypeScript, ethers v6, plain CSS	The wizard, vault page, Attack Lab and Verify panel. Fonts and scripts are bundled so the page makes no external requests.
`shared/`	TypeScript, zod, ethers	The checklist schema, canonical JSON, hash helpers, validator and gate. This is the single place the agreement hash is defined.
`samples/`	Text and JSON	Fictional agreement text, the seeded confirmed checklist, and demo deliverable files.

4.2 Why the boundary is a type, not a convention

The checklist exists in two forms in the code. A `ProposedChecklist` is what the extractor returns and may contain anything. A `ConfirmedChecklist` exists only after the gate has accepted it. The function that deploys a vault takes a confirmed checklist, recomputes its hash, and reads the stored hash and total

back from the chain after deployment. A small file, `deploy.boundary.ts`, contains a line that must fail to compile: it tries to pass a proposed checklist to the deploy function. If someone ever loosens the types so that line compiles, the type-check step fails and tells them.

4.3 The agreement hash

The hash is keccak256 of the UTF-8 bytes of the confirmed checklist in canonical form. Canonical JSON sorts object keys recursively using the default UTF-16 order, uses no whitespace, keeps arrays in order, and writes amounts as decimal wei strings. The serialiser throws on values that have no single canonical form, such as `undefined`, big integers, `NaN` and non-plain objects. There is exactly one implementation, in `shared/src/canonical.ts`, and a frozen test vector with deliberately unsorted keys and non-ASCII text. We cross-checked that vector's canonical string against an independent Python implementation. The confirmed JSON is stored by the server under its hash, so anyone can re-hash it later using the Verify panel.

5 Contract design

One `MilestoneVault` deployment is one agreement. The client, provider and arbiter are immutable, non-zero and pairwise distinct. The agreement hash, review window and total amount are immutable too. An agreement holds between one and twenty milestones. The specification in `docs/04-CONTRACT-SPEC.md` is the source of truth for everything in this section.

5.1 State machines

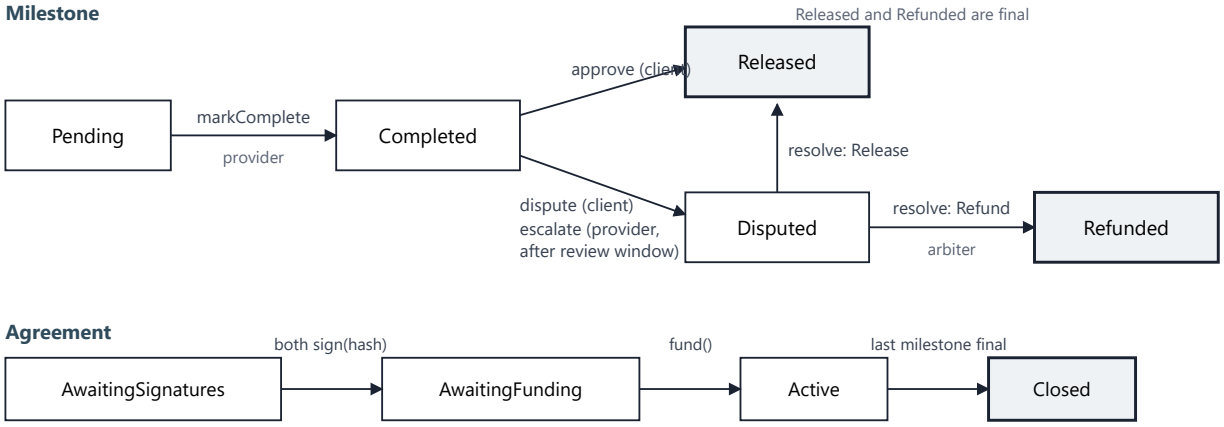


Figure 2. Every transition not drawn here reverts with a named custom error.

5.2 Transition table

From	Function	Caller	Extra checks	Effect
Pending	`markComplete(i, deliverableHash)`	Provider	Agreement active, `i` equals the next milestone, hash not zero	Completed
Completed	`approve(i, deliverableHash)`	Client	Hash equals the stored hash	Released, provider credited
Completed	`dispute(i, evidenceHash)`	Client	None	Disputed
Completed	`escalate(i)`	Provider	Review window has elapsed	Disputed
Disputed	`resolve(i, Release, rationaleHash)`	Arbiter	None	Released, provider credited
Disputed	`resolve(i, Refund, rationaleHash)`	Arbiter	None	Refunded, client credited

Other functions are `sign` (client or provider, once each, with the right hash), `fund` (client, exact total, payable), `withdraw` (anyone with a credit, non-reentrant), and the read-only `reconcile`, `milestoneCount` and `getMilestone`. Both `receive` and `fallback` revert with `DirectPaymentNotAccepted`.

Checks inside every function run in a fixed order: role, then agreement state, then index, then milestone state, then any extra checks. We made that order part of the specification so the Attack Lab always reports the first rule that failed, and so a reviewer reading a revert knows which layer refused.

5.3 Events and errors

Every state change emits an event, and the web timeline is built from them: `AgreementSigned`, `AgreementFunded`, `MilestoneCompleted`, `MilestoneApproved`, `MilestoneDisputed`, `MilestoneEscalated`, `DisputeResolved`, `Credited`, `Withdrawn` and `AgreementClosed`. There are no revert strings. Failures use custom errors that carry the values needed to explain them, for example `InvalidMilestoneState(index, actual)`, `MilestoneOutOfOrder(index, expectedIndex)`, `DeliverableHashMismatch(expected, provided)` and `ReviewWindowOpen(escalationAllowedAt)`.

5.4 Reconciliation

The `reconcile()` view returns what was deposited, released, refunded and withdrawn, what is held in escrow and by open milestones, what is owed to the parties, the balance the books expect, the balance the contract actually holds, and any surplus. Its `ok` flag is true when the real balance is at least the expected balance, escrow equals the sum of open milestones, and what is owed equals released plus refunded minus withdrawn. Every subtraction in it is guarded so that the view itself never reverts, even if the balance were somehow lower than expected, in which case it reports `ok = false`.

5.5 Invariants

These are asserted after every step of the randomised test described in section 8.

I1	The real balance is at least the expected balance, and equal unless ether was force-sent, in which case the difference is reported as surplus.
I2	The deposited total is either zero or exactly the agreed total.
I3	Funds held in escrow equal the sum of milestones that are still pending, completed or disputed.
I4	The released and refunded totals equal the sums of the milestones in those states.
I5	Amounts owed to the parties equal released plus refunded minus withdrawn.
I6	A final milestone never changes state, and a failed call changes no storage.
I7	Every milestone beyond the next one is still pending.
I8	The arbiter's credit is always zero. The arbiter can never be paid.

6 Security design

The table lists the main threats we designed against, the mechanism that answers each, and where a reader can see it demonstrated. "Test" refers to the Hardhat suites in `contracts/test/`.

Threat	Mechanism	Evidence
Releasing the same milestone twice	Released and Refunded are final states. Credit is added once, on the transition.	Test *rejects a second release attempt*; Attack Lab
Refund by someone not entitled	Only `resolve` can refund, only the arbiter can call it, and only for a disputed milestone.	Test *rejects an unauthorized refund*; Attack Lab
Skipping or reordering milestones	`nextMilestone` tracks the only milestone that may be completed; every function checks state first.	Test *rejects an out-of-order transition*
Re-entrancy on withdraw	Checks-effects-interactions (credit zeroed and totals updated before the external call), plus OpenZeppelin `nonReentrant`.	Test *blocks a re-entrancy attack*: every re-entry attempt is blocked, none succeeds, and the attacker is paid exactly the amount owed. The same pattern drains the contrast contract.
A swapped deliverable	`approve` requires the exact stored hash.	Test *rejects approval of a swapped deliverable*; vault page
Changing the agreement after the fact	The agreement hash is immutable, and signing requires that exact hash.	Tests on signing a different hash and on double signing
Ether sent straight to the contract	`receive` and `fallback` revert.	Test *rejects direct ETH transfers, with or without calldata*
Ether forced in by self-destruct	Accounting uses its own counters. Extra ether appears as surplus and can never be withdrawn.	Test with `ForceSender`; Attack Lab
A recipient that refuses payment	Pull payments. If a transfer fails, the credit and the books are restored.	Test with `RejectingProvider` (`TransferFailed`)
Paying the arbiter, or party collusion by address reuse	The three addresses are distinct, and only client and provider can hold credit.	Constructor tests; invariant I8

The contract uses no `tx.origin`, no `delegatecall` and no `selfdestruct`. The one `selfdestruct` in the repository is in `ForceSender`, a test-only contract that exists to prove the surplus accounting, and the compiler warns about it. The vault is not upgradeable, and there is no owner or admin role that can move funds.

7 The AI component

7.1 What AI does, and what it does not

The model turns agreement prose into a proposed checklist of parties, milestones, amounts, conditions and deadlines. It also classifies ambiguities by type and quotes the sentence that caused each one. It does not write to the contract, decide whether money moves, or recommend a ruling. The pipeline is: the model proposes, the validator checks, a person resolves and confirms, the result is hashed, both parties sign, and then the vault is deployed and funded.

7.2 Providers and fallback

The service picks a provider in this order: a local Ollama model if it answers within 1.5 seconds and the configured model is installed, then Gemini if an API key and model name are configured, then the built-in rules extractor, which is always available and deterministic. Model output is parsed with zod; on a parse failure the service retries once with the error appended, and falls back to rules if that also fails. Every response reports which source produced it, the model, the latency and the reason for any fallback. If the service itself is unreachable, the web app runs the same rules in the browser so the wizard still works, and says so on screen.

After parsing, the rules overwrite some of what the model said. The ambiguity identifier, its severity, its origin and each condition's "objective" flag are set by the rules, not the model, and the milestone index is simply the position in the list. A quote the model gives that cannot be found in the source text, allowing for case, spacing and curly quotes, is replaced with nothing and noted.

7.3 Ambiguity types

Type	Severity	Typical trigger
`MISSING_PARTY`	Blocking	No arbiter is named
`PARTY_CONFLICT`	Blocking	One person fills two roles
`MISSING_AMOUNT`	Blocking	"Payment on delivery" with no figure
`AMOUNT_MISMATCH`	Blocking	Milestones add up to 4.5 but the stated total is 5
`CONDITIONAL_AMOUNT`	Blocking	"Remaining funds", "bonus if early", "10% of"
`AMOUNT_NOT_IN_TEXT`	Blocking	An amount that does not appear in the source (guard against invented figures)
`UNDEFINED_TRANSITION`	Blocking	"Released automatically if no reply"
`CURRENCY_UNCLEAR`	Blocking	"Fifty thousand rupees or equivalent"
`VAGUE_CONDITION`	Blocking, or warning	"Satisfactory", "reasonable", "as needed". Blocking when a milestone has no objective condition.
`SUBJECTIVE_APPROVAL`	Warning	"To the mentor's satisfaction"
`MISSING_DEADLINE`	Warning	A milestone with no date

7.4 Validator and gate

The validator in `shared/` runs on every proposal, whatever produced it, and the server and the browser run identical code. It checks that all three parties are present and distinct, that there are between one and twenty milestones, that every amount is present and positive, that the milestone amounts sum exactly to the stated total (compared in wei), that each milestone has at least one objective condition, and that the wording does not contain vague, conditional or automatic-transition phrases. It also checks grounding: every non-null amount must appear as "number ETH" in the source text, and every quote must be a substring of it. The validator may add or escalate a flag, but it never removes one the model raised.

The Confirm button is enabled only when four conditions hold. Every ambiguity has a resolution. Every blocking ambiguity was resolved by editing or removing the clause, since acknowledging it is not enough. Re-running the validator on the edited checklist finds no blocking issue, because an edit can introduce a new one. And each of the three roles has a distinct address. The server recomputes what must be resolved from the original proposal and source text, so a caller that submits an empty ambiguity list cannot skip the first two conditions, and a test covers that case.

7.5 What we have measured so far

We ran a real local model, `qwen2.5:7b-instruct`, on the KrishiLens text. It extracted the parties, total, titles, amounts and deadlines correctly. A cold first call took 26.8 seconds against a 30 second timeout, a warm call took 19.6 seconds, and the wizard run recorded 25.3 seconds. The model also raised two flags beyond the expected five. One was a reasonable vague-condition warning. The other, `UNDEFINED_TRANSITION` on the third milestone, was a false positive, because nothing in the text releases money automatically. A person had to clear it by removing the clause. We kept that behaviour on purpose: a false alarm costs one click, whereas a missed ambiguity can cost the agreement.

> **Not yet available.** The planned evaluation (prompt P06), which would report precision, recall and false-positive rate for the rules extractor against the language model on labelled agreements, has not been run. It depends on the teammate's set of sample agreements and expected labels, which are not yet written. This report therefore quotes no accuracy figures for the AI component, only the single recorded run above.

8 Verification evidence

The figures below are those recorded in `docs/PROGRESS.md` on 11 October 2026.

Suite	Result	What it covers
Contract tests (Hardhat)	138 passing	Constructor validation, lifecycle, access control tables, a state machine table of six functions against five milestone states, the adversarial suite, and the attack contracts. Includes the four booklet-required tests by exact name.
Invariant fuzz	5 seeds x 200 steps	1,000 steps over 21 randomly shaped vaults. 401 calls succeeded and 599 reverted. Thirteen distinct custom errors were hit, no revert was a panic or empty, every success emitted an event, I1 to I8 held after each step, and failed calls left storage unchanged.
Coverage	Lines 100% Branches 98.15%	`MilestoneVault.sol`, with test contracts excluded. The two uncovered branches are unreachable defensive guards inside `reconcile()`.
Shared tests (vitest)	115 passing	Canonical JSON, the frozen hash vector, the strict schema, file hashing, validator rules V1 to V11, merging, gate rules, and consistency of the sample files.
Server tests (vitest)	42 passing	The rules extractor, extract, fallback, retry and grounding using fake Ollama and Gemini servers, validate and gate-check routes, confirmation, route hardening, and a check that the server contains no chain access.
Web tests (vitest)	62 passing	Wizard state and gate wiring, revert decoding, the vault model across five states and four roles, timeline grouping, reconciliation diagnosis, and terms verification. The type-check and production build are clean.
End-to-end (real Chrome)	Three scripts pass	The wizard, the vault dashboard (on the seeded vault and on one created by the wizard), and the Attack Lab. Run on a clean reset.

8.1 The four required tests

The booklet's four adversarial tests exist under these exact names in `contracts/test/Adversarial.test.ts`: *rejects a second release attempt*, *rejects an unauthorized refund*, *rejects an out-of-order transition* and *blocks a re-entrancy attack*. The same file also holds further named guarantees, such as rejecting a wrong funding amount, early escalation (including one second before the boundary), and withdrawing twice.

8.2 The Attack Lab

The Attack Lab turns the adversarial tests into something a judge can operate. It offers twelve attacks and one contrast, each a real transaction sent from sandbox accounts. Every attack deploys its own fresh three-milestone vault, so no result depends on a previous one and the demo vaults are never touched. An end-to-end check compares the seeded vault's events, state, balance and code before and after 39 sandbox runs to confirm this.

The lab reports one of three outcomes: a pass (the contract did what was expected), FAILED (the contract did something else), or a lab error (the lab could not finish). Only a pass counts as blocked, and the score is computed from the results. The contrast row runs the same re-entrancy pattern against the

deliberately vulnerable `NaiveEscrow`, which loses 3 ETH, to show that the attacker is real and that the real contract's refusal is meaningful. Three consecutive "Run all" executions without a page reload each blocked twelve of twelve with the ledger reconciled after every attack.

9 Requirements traceability

Each requirement from the booklet, where it is met, and what a judge can look at. Identifiers follow `docs/01-PROBLEM.md`.

ID	Requirement	Where it is met	What can be inspected
R1	Solidity contract on local Hardhat	`MilestoneVault.sol` on a Hardhat node	Live demo; `npm run test:contracts`
R2	Client, provider and arbiter named	Immutable constructor parameters, distinct and non-zero	Constructor tests
R3	Per-milestone amount and conditions	Amounts on-chain; conditions in the confirmed checklist whose hash is on-chain	Verify agreement panel
R4	Provider marks complete	`markComplete`, provider only	UI and test
R5	Client approves, releasing funds	`approve`, client only, hash must match	UI, test, swapped-file attack
R6	Client disputes to the arbiter	`dispute`, client only	UI and test
R7	Arbiter releases or refunds	`resolve`, arbiter only	UI and test
R8	Each step limited to its party	Role modifiers	Attack Lab, wrong-role rows
R9	Undefined transitions rejected	`InvalidMilestoneState` and related errors on every function	Attack Lab, out-of-order row
R10	No duplicate release	Final states; credit applied once	Test and Attack Lab
R11	No unauthorised refund	Only `resolve`, only the arbiter, only when disputed	Test and Attack Lab
R12	Balance always reconciles	`reconcile()`, live ledger, randomised invariant test	Ledger stays green through every attack
R13	The four required tests	`Adversarial.test.ts`, exact names	Test output
R14	AI proposes a checklist with ambiguities	Extract route (Ollama, Gemini or rules)	Wizard step two
R15	A person confirms; ambiguities resolved	Gate rules 1 to 4, enforced on server and in the browser	Confirm stays locked; tests
R16	AI output never written directly	Proposed and confirmed types; deploy accepts only a confirmed, re-hashed checklist	Code walkthrough; boundary file
C1	AI used meaningfully	Extraction and ambiguity classification	One recorded live run. Evaluation (P06) pending
C2	Local fallback	Rules extractor, local Ollama, in-browser rules, bundled assets	Server-down and no-Ollama walkthroughs
C3	Reproducible live testing	`npm run demo:reset`, seeded scenario, Attack Lab sandbox	Judges operate it themselves
C4	Fictional data	Fictional names and test ether only	`samples/`

10 Project status

Work package	Status	Notes
P00 Bootstrap	Done	Workspaces, scripts, toolchain checks
P01 Contract core	Done	Implemented to the specification
P02 Core tests	Done	Mutation check performed by hand
P03 Attacks and invariant test	Done	Re-entrancy blocked, naive contrast drained, force-send surplus, seeded fuzz
P04 Deploy and seed	Done	Shared hashing with a frozen vector, deploy and seed scripts, ABI export, idempotent reset
P05 AI service	Done	Validator, gate, rules extractor, Ollama and Gemini providers, server routes
P06 AI evaluation	Open	Blocked on the teammate's sample agreements and labels
P07 Wizard	Done	Walked in rules, live Ollama and server-down modes
P08 Vault page and ledger	Done	Milestone actions by role, timeline, ledger, Verify
P09 Attack Lab	Done	Twelve attacks and one contrast
P10 Stretch features	Open	Arbiter evidence brief, git-commit deliverables, vault factory
P11 Integration and README	Open	No project README exists yet
P12 Judge-proof audit	Open	Final review and rehearsal

The core of the product (contract, tests, validator, wizard, vault page and Attack Lab) is working. What is missing is the measured evaluation of the AI component, the integration pass that produces a README and a one-command demo, and the final audit.

11 Limitations and known issues

We would rather state these plainly than have a reviewer find them.

- **It is a prototype on a local test chain.** Roles are switched using Hardhat's unlocked accounts, there is no wallet extension, and nothing has been deployed to a public network. The project has had no external security audit.
- **The AI is only partly measured.** There is one recorded live run and no accuracy statistics. With a live model, false positives such as the one described in section 7.5 must be cleared by a person.
- **Model latency is close to its timeout.** A cold local model took 26.8 seconds against a 30 second limit and could fall back to rules. A warm-up call at server start is planned.
- **Long agreements are handled unevenly.** The extract call grounds and validates against the first 6,000 characters, while the gate and the source hash use the whole text. The web interface therefore drives its cards from the gate check, not from the extract response.
- **An extreme review window can panic.** The escalation time is computed as `completedAt + reviewWindow` in checked 64-bit arithmetic, so a value near 264 would revert with a panic instead of a custom error. A cap in the constructor would fix it; the specification has not been changed.
- **Gemini has not been tested against the real service,** because no key was available. It is covered only by tests against a fake server.
- **Test-only code.** `ForceSender` uses `selfdestruct` and the compiler warns about it. It is not part of the product.
- **Tooling notes.** The production web bundle is a single 646 kB chunk, and the dependency install reports audit warnings in transitive development packages of the Hardhat toolchain. On the Windows development machine, local loopback connections were intermittently refused, which occasionally made an end-to-end run fail until it was repeated.
- **A testnet configuration exists but is not used.** A network entry for the MST testnet is registered only when a deployer key is present in a local environment file. No deployment has been made, and the web app is not set up to use it.

12 Roadmap and open decisions

Everything in this section is proposed and not approved scope. The order of work stays as planned: finish the AI evaluation, then integration and the audit, and only then any stretch work. None of the ideas below changes the contract.

Idea	What it adds	Why it matters
A	A preflight on every action button, and a free-form "try your own attack" panel in the sandbox.	Judges see the refusal before any transaction is sent, and can attack the contract in their own way. Builds on the refusal decoding and Attack Lab that already exist.
B	A deliverable manifest, with a hash per file and one hash over the canonical list.	The contract still stores one value, but a mismatch can name the file that changed. The client needs the provider's manifest to see which one.
C	A seal receipt and a standalone verifier.	A single file recording the terms, the on-chain signing events and the deliverable hashes, which can be checked without the web app. It records the signing transactions, because the signatures are on-chain calls.
D	An arbiter evidence brief (already in the stretch plan).	Lines up each side's evidence against each condition for the arbiter, with no recommended ruling, and with a rules fallback.
E	The AI evaluation with a red-team set.	Precision, recall and false-positive rate, including agreements that try to inject instructions.
F, G	Automated mutation testing, and a static analysis run (Slither) reported in the README.	Stronger evidence for the contract than coverage alone.
H, I	Plainer wording in the interface, and a recorded backup of the demonstration.	Clearer labels with the hashes still visible in detail views, and protection against a failure during a live demonstration.

Decisions that need an owner

1. Whether the brand is written *Sealstone* or *SealStone*, and whether to sweep the remaining "MilestoneVault" wording from the interface and documents (contract identifiers would keep the booklet's name).
2. Whether a public testnet deployment counts as a "live third-party system" under the booklet's data rule. The booklet does not require one, and our own rule says no public networks by default, so this is worth asking the organisers.
3. Whether to add a path for an arbiter who never rules. It would change the contract specification, and the existing escalation already covers a client who stays silent.
4. Which of the ideas above to adopt, and in what order.

13 Related work

We looked at escrow and attestation projects from hackathons and grant programmes. We separate confirmed winners from projects whose placement we could not verify, and we only call something a winner when a source says so.

Project	Standing	Approach, and how Sealstone differs
Agent Escrow	1st place, APU Web3 AI hackathon	Every stage enforced on-chain, with a reentrancy guard. Pays automatically after a review period. Sealstone escalates to the arbiter instead, because an automatic payment is an undefined transition.
Chainlink Convergence 2026 winners	Hackathon winners	Check actions before they execute. We take the framing as a gate in front of money, applied to agreement text and contract calls.
Vernari Protocol, LinkPal	Winners, earlier Chainlink hackathons	Escrow released on verified outside data, through oracles. Sealstone releases against a hash that the client checks, which keeps it working offline.
Attestify, Aivy	Winners, Hedera Apex	Attestations and agent escrow. We note them as influences on the seal receipt idea.
Trustless Work, Boundless	Grants from the Stellar Community Fund	Role-separated and milestone-based escrow. These are funding awards, not hackathon wins. We borrowed the role model and the "submit proof" step.
TaskLoop, WorkProof, ArbitrAI, TrustArc AI Escrow, AI-Trustless Escrow	Placement not confirmed	These let AI review work or rule on disputes, sometimes with a confidence threshold or a vote of several models. Sealstone does not allow that, since the booklet forbids AI output deciding or being written to the contract.

The combination we believe is distinctive is the pairing of an ambiguity gate that blocks funding until a person has resolved every flagged term, a client approval bound to an exact deliverable hash, an Attack Lab that visitors can operate against a deliberately vulnerable contrast, and a live reconciliation that surfaces force-sent ether. We did not find that combination in the projects we reviewed, but we reviewed only a limited number and cannot claim it is unique.

Sources

docs/03-REFERENCES.md (project reference list and its cited URLs, including apu.edu.my/node/3306, communityfund.stellar.org and ethglobal.com/showcase pages); Hedera Hello Future Apex winners (hedera.com/blog/these-are-the-winners-of-the-hello-future-apex-hackathon); Chainlink Spring 2023 winners (blog.chain.link/spring-2023-hackathon-winners); Chainlink Virtual Hackathon winners (blog.chain.link/winners-of-the-chainlink-virtual-hackathon); AI-Trustless Escrow (ethglobal.com/showcase/ai-trustless-escrow-eg79v); ArbitrAI (chain.link/hack-26/projects/arbitrai); WorkProof (hackquest.io/en/projects/WorkProof); TrustArc AI Escrow (lablab.ai); VORTEXA 3.0 Problem Statement Booklet, page 11 and common rules.

Appendix A Commands and repository layout

Commands

The host machine runs Windows, so the npm scripts avoid shell-specific syntax.

`npm run setup`	Install all workspaces
`npm run chain`	Start the Hardhat node on port 8545
`npm run deploy:demo`	Deploy and seed the demonstration scenario, and write the generated files used by the web app
`npm run test:contracts`	Run the Hardhat test suites
`npm run coverage`	Run solidity-coverage
`npm run server`	Start the AI service on port 8787
`npm run web`	Start the web app on port 5173
`npm run dev`	Run chain, server and web together
`npm run demo:reset`	Reset the chain and reseed for a clean demonstration
`npm run eval:ai`	Planned (P06): extraction and ambiguity evaluation over the samples

Repository layout

`contracts/contracts/`	`MilestoneVault.sol` and `attack/`
`contracts/test/`	Constructor, lifecycle, access control, state machine, adversarial and invariant suites
`server/src/`	The Express app and the `ai/` providers
`shared/src/`	Canonical JSON, hashing, schemas, extractor, validator and gate
`web/src/`	Pages (wizard, vault, Attack Lab), libraries, UI components and generated contract data
`web/e2e/`	Real-browser end-to-end scripts
`samples/`	Agreement text, the seeded confirmed checklist, demo deliverable files
`docs/`	Problem statement mapping, plan, references, contract and AI specifications, demo script, UI notes, progress log, and the ideas backlog

This report is generated from `docs/report/sealstone-report.html`. Update the figures against `docs/PROGRESS.md` before reissuing it.